

설비 고장을 예측하는 AI 데이터 분석

예외처리 및 종합 실습

견고한 예외처리

PART 2

견고한 예외처리

else와 finally부터 with open,
bare except 회피, raise 검증까지
실무에서 통하는 예외처리 패턴을 정리합니다.

01 else와 finally 구성

else와 finally / 리소스 정리

02 안전한 파일 처리 구성

with open 활용 / CSV 예외처리

좋은 예외처리 원칙 구성

bare except / raise 활용

이 단원의 실데이터 — ICT 검사

예외처리로 다룰 현장 검사 데이터를 먼저 이해

공정 — ICT 검사

ICT는 제조 부품의 전기 특성을 측정해 규격 안에 드는지 판정하는 검사 공정이다.

핵심 값과 결측

부품명별로 측정값·기준값·상한/하한을 기록한다. 실제 현장 데이터에는 빈칸(결측)·형식 오류가 섞여 있다.

왜 예외처리

측정값이 비거나 문자·범위 밖이면 형변환·인덱스 오류가 난다. try·except로 불량 줄을 건너뛰고 견고하게 처리한다.

else와 finally

else는 성공했을 때만 실행되는 블록

finally는 무조건 마지막에 실행되는 블록

지난 시간에 배운 **try / except**에 두 형제가 더 붙습니다.

else는 조건문의 else와 같은 단어로, try에서 예외가 하나도 발생하지 않았을 때만 실행됩니다.

finally는 '마지막으로'라는 뜻 그대로, 성공하든 실패하든 무조건 마지막에 실행됩니다.

이 성질 덕분에 finally는 **파일 닫기처럼 반드시 처리해야 할 마무리 작업**에 자주 쓰입니다.

else와 finally 코드

CODE

무조건 finally 실행

```
try:
    temp = float(text)
finally:
    print("종료")    # 성공·실패 무조건 실행
```

'62.5'를 넣으면 성공 후 '종료', 'abc'를 넣으면 실패 후 '종료'가 출력됩니다.

성공이든 실패든 finally는 양쪽 다 실행되는 것이 핵심입니다.

네 블록의 실행 정리

try·except·else·finally — 네 블록이 어떤 경로로 실행되는지 정리합니다.

CASE 01

성공 경로

try를 끝까지 실행한 뒤
else로 이동하고,
마지막에 **finally**가 실행됩니다.

CASE 02

실패 경로

try가 중단되면
except로 이동하고,
마지막에 **finally**가 실행됩니다.

CASE 03

자주 쓰는 것

넷을 다 쓸 필요는 없습니다.
초보 단계에서는
finally가 가장 유용합니다.

리소스 정리가 필요한 이유

열어둔 파일을 닫지 않으면 **데이터 손실과 자원 낭비**
정리는 **선택이 아니라 의무**

파일을 열면 운영체제가 우리 프로그램에 **연결 통로**를 내어줍니다.

이 통로는 **한정된 자원**이라서, 닫지 않고 계속 쌓이면 결국 고갈됩니다.

게다가 파일에 쓴 내용은 닫는 순간 실제로 저장되기 때문에, 닫지 않으면 **저장 대기 중인 데이터가 사라질 수 있습니다.**

그래서 파일 닫기는 '하면 좋은 것'이 아니라 **반드시 해야 하는 의무**입니다.

파일을 안 닫으면 생기는 문제

안 닫힌 파일이 어떤 결과를 만드는지 세 가지로 정리합니다.

RISK 01

통로 고갈

파일을 계속 열고 닫지 않으면
운영체제가 더 이상
통로를 내주지 못하는 오류가
발생합니다.

RISK 02

데이터 손실

저장 대기 중인 내용이
그대로 사라질 수 있습니다.
파일은 닫는 순간
실제로 저장됩니다.

RISK 03

예외 시 위험

읽다가 예외가 나면
그 아래 **close**가 **실행되지 않습니다**.
그래서 finally에
닫기를 넣어야 합니다.

실습 1. finally로 파일 안전하게 닫기

try-finally로 오류가 나도 파일을 반드시 닫기

단계

- ① try 블록에서 파일을 열어 처리
- ② 처리 도중 오류가 날 수 있음을 가정
- ③ finally 블록에 close를 넣어 오류 여부와 상관없이 닫기
- ④ 일부러 오류를 내도 finally가 실행되는지 확인

예상 결과 오류가 나도 파일이 안전하게 닫힘

with open으로 더 안전하게

with open을 쓰면 자동으로 닫힘 — 파일 처리의 **표준**

CODE

블록이 끝나면 자동으로 닫힘

```
with open(name) as f:
    data = f.readlines()    # 블록 끝나면 자동으로 닫힘
```

with open as f 형태로 쓰면, 들여쓴 블록이 끝나는 순간 파이썬이 알아서 파일을 닫아줍니다.

심지어 블록 안에서 예외가 나도 빠져나가기 전에 반드시 닫아줍니다. **finally**를 직접 쓴 것과 같은 효과입니다.

앞으로 파일을 열 때는 **with open**을 **기본으로** 사용합니다.

with open 사용 시 주의점

with open이 자동화해 주는 부분과, 여전히 직접 챙겨야 할 부분을 정리합니다.

POINT 01

자동 닫힘

블록이 끝나면,
예외가 나도 **무조건 닫힙니다.**
닫기를 자동화해 주는
가장 큰 장점입니다.

POINT 02

블록 밖 주의

블록 밖에서 쓸 값은
블록 안에서 **리스트나 변수에**
담아두어야 합니다.
밖에서는 이미 닫혀 있습니다.

POINT 03

열기 실패 대비

with는 닫기만 자동입니다.
파일 없음은
여전히 try-except로
따로 처리해야 합니다.

실습 — CSV 읽기 정답

CODE

값 변환과 파일 없음 대비

```
try:
    add(float(row_temp))
except ValueError:
    pass    # 숫자로 못 바꾸는 칸은 건너뛰
```

바깥 try-except는 파일이 없는 경우를 막고,

안쪽 try-except는 각 칸을 숫자로 못 바꾸는 경우를 막습니다.

두 위험을 **각각 다른 위치에서** 처리하는 것이 핵심입니다. 정상 파일은 40개, 불량 파일은 9개가 모이는 이유를 생각해 보세요.

반복문 안에서의 예외처리

반복문 안에 **try-except**를 넣고 **continue**로 불량 줄만 건너뛰기

대용량 데이터를 안전하게 처리하는 **핵심 패턴**

가장 실용적인 패턴입니다. 수천 줄에서 **불량 줄만 쏙 빼고** 나머지는 모두 처리합니다.

데이터를 한 줄씩 반복하며 각 줄의 처리를 try로 감싸고, 문제가 생기면 except에서 **continue**로 다음 줄로 넘어갑니다.

이 패턴 덕분에 5천 줄 중 불량 몇 줄만 빠지고 나머지는 정상 처리됩니다. **데이터를 두려워하지 않게 만드는 무기**입니다.

반복문 예외처리 코드

CODE

불량 줄은 continue로 건너뛰기

```
try:
    temp = float(row_temp)
except ValueError:
    continue    # 불량 줄은 건너뛰고 다음 줄로
```

각 줄의 변환을 try로 감쌌습니다. 한 줄에서 예외가 나도 **그 줄만 영향**받습니다.

except에서 불량을 센 뒤 **continue**로 다음 줄로 넘어갑니다. 반복은 멈추지 않습니다.

try를 반복문 **안쪽에** 두는 것이 핵심입니다. 바깥에 두면 첫 불량에서 전체가 멈춥니다.

반복문 예외처리 포인트

반복문 예외처리에서 자주 놓치는 세 가지 포인트를 짚어 봅니다.

POINT 01

continue 역할

이번 회차를 멈추고
다음 회차로 넘어가라는
명령입니다.
'이 줄 건너뛰고 다음 줄로.'

POINT 02

try 위치

반복문 안쪽에 두어야
불량 줄만 건너뛴니다.
바깥에 두면
첫 불량에서 전체가 중단됩니다.

POINT 03

불량 기록

줄 번호를 모아두면
나중에 어떤 데이터가
불량이었는지
점검할 수 있습니다.

실습 2. 반복문에서 불량 줄 건너뛰기

반복문 안에서 try-except와 continue로 불량 줄만 건너뛰기

단계

- ① 여러 측정값(일부는 숫자가 아님)을 반복
- ② try에서 float로 변환
- ③ 변환 실패(ValueError) 시 continue로 그 줄만 건너뛰기
- ④ 정상 값만 합계에 더해 출력

예상 결과 불량 줄은 건너뛰고 정상값만 합산

실습 3. 여러 파일 묶어 처리하기

여러 파일을 묶어 처리하되 없는 파일은 건너뛰기

단계

- ① 여러 파일 이름을 반복
- ② try에서 파일을 열어 처리
- ③ 없는 파일(FileNotFoundError) 시 continue로 건너뛰기
- ④ 처리한 파일 수를 세어 출력

예상 결과 없는 파일은 건너뛰고 있는 파일만 처리

bare except의 위험

모든 예외를 뭉뚱그려 잡는 except는 **진짜 버그를 숨김**

잡을 예외는 **구체적으로 지정**

except 뒤에 아무것도 안 붙이고 그냥 except만 써도 동작합니다. 모든 예외를 다 잡는 이것을 **bare except**라고 부르며, 피해야 할 나쁜 습관입니다.

bare except는 예상한 예외뿐 아니라 **변수 오타 같은 진짜 버그**까지 잡아 조용히 넘어가 버립니다. 그러면 버그가 있는지조차 모르게 됩니다.

except 뒤에는 반드시 **ValueError**처럼 **구체적인 이름**을 적으세요.

bare except를 피해야 하는 이유

bare except가 위험한 두 가지 이유와, 대신 해야 할 한 가지를 정리합니다.

RISK 01

버그 숨김

변수 오타 등
진짜 버그도 잡아서
못 찾게 됩니다.
경고등을 테이프로 가리는 격.

RISK 02

중단 신호까지

강제 종료 신호마저 잡아
프로그램을
못 멈출 수 있습니다.

DO INSTEAD

대안

예상 예외만 적고,
새 예외를 만날 때마다
하나씩 추가합니다.

좋은 예외처리 원칙

예상한 문제는 **우아하게 처리**

예상 못한 문제는 **정직하게 드러내기**

문법을 아는 것과 잘 쓰는 것은 다릅니다. 실무에서 다져진 좋은 예외처리 원칙을 정리합니다.

핵심은 이것입니다 — **예상한 문제는 우아하게 처리하고, 예상 못한 문제는 숨기지 말고 드러내라.**

이 **균형 감각**이 견고한 프로그램과 위험한 프로그램을 가릅니다.

나쁜 예외처리 vs 좋은 예외처리

세 가지 항목으로 나쁜 예와 좋은 예를 나란히 비교합니다.

구분	나쁜 예	좋은 예
예외 지정	전부 잡기	구체적 지정
try 범위	전체 감쌈	위험한 줄만
문제 처리	무시	줄 번호 기록

좋은 예외처리 4원칙

구체적으로 · 작게 · 기록 · 드러내기 — 네 단어로 정리되는 원칙입니다.

01 구체적으로

bare except 대신 **예상되는 예외만**
정확히 지정합니다.

02 작게

위험한 코드만 try에 담아
오류 위치를 명확히 합니다.

03 기록

어떤 데이터가 왜 빠졌는지 기록해야
분석 신뢰도가 올라갑니다.

드러내기

예상 못한 예외는 잡지 말고 드러내서
코드를 고치게 돕니다.

에러 정보 꺼내 쓰기 — as e

CODE

as e로 원인 메시지 활용

```
try:
    temp = float(v)
except ValueError as e:
    print(e)    # e에 예외 원인 메시지
```

except 뒤에 **as e**를 붙이면 **e에 예외 정보**가 담깁니다.

e를 출력하면 트레이스백 맨 아랫줄에서 봤던 그 **구체적인 설명**이 나옵니다.

단순히 '문제 발생'이 아니라 **어떤 값이 왜 문제인지**를 기록할 수 있습니다.

as e 활용 포인트

e가 주는 원인 정보를 어떻게 활용할지 세 가지 관점으로 정리합니다.

POINT 01

맥락 결합

내가 아는 **줄 번호**와
파이썬이 주는 **원인**을 합치면
더 정확한 기록이 됩니다.

POINT 02

사용자용

사용자에겐
'온도 값이 올바르지 않습니다'
처럼 이해하기 쉬운
우리말 안내를 제공합니다.

POINT 03

개발자용

기록에는
e의 구체적 원인을 남겨
나중에 원인을 추적합니다.

raise로 직접 예외 발생시키기

CODE

규칙 어긋 값에 직접 예외 발생

```
if temp > 200:  
    raise ValueError("비정상")    # 우리 규칙을 벗어나면 직접 예외 발생
```

영하 999도 같은 값은 파이썬 입장에선 정상 숫자입니다. 아무 예외도 나지 않습니다.

하지만 우리는 이 값이 **센서 오류**인 것을 알고 있습니다.

우리가 정한 범위를 벗어나면 **raise**로 직접 ValueError를 일으킵니다. 이렇게 일으킨 예외도 try-except로 잡을 수 있습니다.

raise는 **파이썬이 못 잡는 우리만의 규칙**을 코드로 강제하는 도구입니다.

raise 활용 포인트

raise를 실무 검증 함수에 활용할 때 짚어야 할 세 가지 포인트입니다.

POINT 01

현실 규칙

온도·압력·진동마다
현실적인 **정상 범위**가
있습니다.
raise로 이 범위를 강제합니다.

POINT 02

역할 분리

검증 함수는 **raise**로
예외만 일으키고,
호출하는 쪽이 **try-except**로
대응을 정합니다.

POINT 03

예외 종류

값이 부적절한 경우이니
ValueError를 쓰는 것이
자연스럽습니다.

실습 4. 함수 안에서 입력값 검증하기

함수 안에서 잘못된 입력을 직접 검증하고 안전하게 처리하기

단계

- ① 입력값을 받는 함수를 정의
- ② try에서 float로 변환해 검증
- ③ 변환 실패 시 except로 안내하고 기본값 처리
- ④ 정상·비정상 입력을 각각 넣어 확인

예상 결과 정상 입력 → 값 / 잘못된 입력 → 안내 후 기본값